

文章笔记

# Claude Code vs. Codex: The Definitive Guide

五道口纳什

2026 年 3 月 29 日

原文作者: hesam (Hesamation)

发布日期: 2026-03-10

阅读时长: 约 12 分钟阅读

原文链接: <https://x.com/Hesamation/status/2031418875946958915>

# 目录

|                                            |          |
|--------------------------------------------|----------|
| <b>1 引言</b>                                | <b>2</b> |
| <b>2 底层模型对比：Opus 4.6 vs. GPT-5.3-Codex</b> | <b>2</b> |
| 2.1 Task-Completion Time Horizon . . . . . | 2        |
| 2.2 Token 经济性 . . . . .                    | 2        |
| 2.3 本章小结 . . . . .                         | 3        |
| <b>3 速度与任务适配性</b>                          | <b>3</b> |
| 3.1 速度不是决定性因素 . . . . .                    | 3        |
| 3.2 任务类型决定胜负 . . . . .                     | 3        |
| 3.3 本章小结 . . . . .                         | 3        |
| <b>4 产品起源与技术栈</b>                          | <b>4</b> |
| 4.1 发展历程 . . . . .                         | 4        |
| 4.2 技术栈对比 . . . . .                        | 4        |
| 4.3 本章小结 . . . . .                         | 4        |
| <b>5 使用体验：“高级开发者” vs. “外包承包商”</b>          | <b>5</b> |
| 5.1 交互风格差异 . . . . .                       | 5        |
| 5.2 市场数据 . . . . .                         | 5        |
| 5.3 本章小结 . . . . .                         | 5        |
| <b>6 RAG Pipeline 实验对比</b>                 | <b>5</b> |
| 6.1 实验设计 . . . . .                         | 5        |
| 6.2 实现方案对比 . . . . .                       | 6        |
| 6.3 行为差异 . . . . .                         | 6        |
| 6.4 评测结果 . . . . .                         | 7        |
| 6.5 本章小结 . . . . .                         | 7        |
| <b>7 生态系统与定价</b>                           | <b>7</b> |
| 7.1 Anthropic 生态的吸引力 . . . . .             | 7        |
| 7.2 定价方案对比 . . . . .                       | 7        |
| 7.3 Skills 与 Plugins 生态 . . . . .          | 8        |
| 7.4 本章小结 . . . . .                         | 8        |
| <b>8 总结与延伸</b>                             | <b>8</b> |
| 8.1 作者的最终选择 . . . . .                      | 8        |
| 8.2 关键结论 . . . . .                         | 8        |
| 8.3 全文核心对比总览 . . . . .                     | 9        |
| 8.4 拓展阅读 . . . . .                         | 9        |

# 1 引言

本文是对 X 平台用户 Hesamation 所撰写的长文 *Claude Code vs. Codex: The Definitive Guide* 的中文整理。作者在使用 Claude Code 数月后转向 Codex，又重新切换回 Claude Code，从模型能力、使用体验、生态系统、定价策略等多个维度对两款 AI 编程代理进行了全面对比，并通过一个 RAG Pipeline 实验提供了定量参考。

## 背景：两款编程代理

**Claude Code** 是 Anthropic 推出的 AI 编程代理，底层模型为 Opus 4.6，支持 1M token 上下文窗口。**Codex** 是 OpenAI 推出的编程代理，底层模型为 GPT-5.3-Codex。两者均提供 CLI 工具和 VS Code 扩展，订阅价格从 \$20/月到 \$200/月不等。

## 2 底层模型对比：Opus 4.6 vs. GPT-5.3-Codex

### 2.1 Task-Completion Time Horizon

衡量模型能力的一个重要指标是 **Task-Completion Time Horizon** (任务完成时间跨度)：给模型一个“人类专家需要  $T$  小时完成”的任务，模型能以多大概率成功完成？

#### 核心对比：任务完成能力差距显著

- **Opus 4.6**：在 50% 成功率下，可处理长达 **12 小时** 的任务
- **GPT-5.3-Codex**：在 50% 成功率下，可处理长达 **5 小时 50 分钟** 的任务
- 在 80% 成功率下，两者差距明显缩小

这表明 Opus 4.6 在处理复杂、长周期任务时具有明显优势。

### 2.2 Token 经济性

基准测试显示两者准确率接近，但 token 消耗差异巨大。Morph 的对比研究发现：

表 1: Token 消耗对比

| 指标            | Claude Code | Codex       |
|---------------|-------------|-------------|
| 相同任务 token 倍率 | 3.2–4.2×    | 1× (基准)     |
| Figma 插件构建    | 6.2M tokens | 1.5M tokens |

#### Token 消耗的实际影响

在相同订阅价格下，Claude Code 用户更可能触及 token 用量上限。如果你的工作流涉及大量代码生成，需要特别关注这一点。

## 2.3 本章小结

Opus 4.6 在长任务上的可靠性显著高于 GPT-5.3-Codex，但 Claude Code 的 token 消耗远高于 Codex。选择时需要在”任务处理深度”和”token 效率”之间权衡。

# 3 速度与任务适配性

## 3.1 速度不是决定性因素

Claude Code 公认速度更快，但作者指出：如果一个代理快速完成任务却留下需要 10 分钟调试的 bug，另一个虽慢却直接交付可用结果，后者的额外时间完全值得。

### 速度陷阱

不要被”我的代理更快”这类说法误导。编程代理是长期使用的工具，**端到端的完成质量**比单次任务的执行速度更重要。

## 3.2 任务类型决定胜负

两款代理在不同编程领域的表现差异很大：

- 在 AI Engineering 任务中，一方可能占优
- 在 Web 开发任务中，另一方可能反超
- 低级编程（low-level programming）领域尚无定论

目前缺乏系统性的跨任务对比研究，且由于模型和代理每隔几个月就会大幅更新，这类研究也很难长期有效。

## 3.3 本章小结

速度优势不等于体验优势，任务类型对两款代理的表现影响极大。理想做法是在自己的实际编程领域中小规模测试两者。

## 4 产品起源与技术栈

### 4.1 发展历程

表 2: Claude Code 与 Codex 发展时间线

| 时间节点 | Claude Code                                                             | Codex                                                   |
|------|-------------------------------------------------------------------------|---------------------------------------------------------|
| 起源   | Anthropic 内部 @bcherny 的 side project，终端原型可调用 Claude API、读取文件、执行 bash 命令 | 最初的 Codex 是 12B GPT-3 微调模型，驱动了初代 GitHub Copilot         |
| 公开发布 | 2025-02-24, Research Preview, 使用 Claude 3.7 Sonnet                      | 2025-04-16, Codex CLI 发布                                |
| 最新版本 | Opus 4.6 (1M token 上下文)                                                 | GPT-5.3-Codex (2026-02-05) ， OpenAI 称其为” 第一个参与了自身创建的模型” |

#### 内部采用速度

Claude Code 原型在 Anthropic 内部发布后，第五天就有一半团队在使用。这种自发的内部采用往往是产品 product-market fit 的强信号。

### 4.2 技术栈对比

表 3: 技术栈对比

| 维度    | Claude Code         | Codex CLI            |
|-------|---------------------|----------------------|
| 编程语言  | TypeScript          | Rust                 |
| UI 框架 | React + Ink (终端 UI) | Ratatui (Rust TUI 库) |
| 分发方式  | 单个 Bun 可执行文件        | 原生二进制                |
| 上下文窗口 | 1M tokens           | —                    |

作者注意到 Claude Code CLI 偶有小 glitch，但不影响实际编码体验。两者本质上都是围绕底层模型 API 的轻量包装 (thin wrapper)。

#### Anthropic 收购 Bun

Anthropic 于 2025 年 12 月收购了 JavaScript 运行时 Bun，Claude Code 因此可以打包为单个 Bun 可执行文件分发，简化了安装流程。

### 4.3 本章小结

Claude Code 源于内部工具的自然生长，技术栈偏向 TypeScript 生态；Codex CLI 选择了 Rust 以追求性能和可移植性。两者都是模型 API 的轻量封装，产品差异更多体现在模型能力和交互设计上。

## 5 使用体验：“高级开发者” vs. “外包承包商”

### 5.1 交互风格差异

开发者社区对两者的体验有一个经典比喻：

体验差异的核心比喻

Claude Code 像一个在你身边工作的高级开发者——会提问、展示推理过程、解释方案。  
Codex 像一个你外包任务后等待交付的承包商——首次尝试准确率高，但交互感较弱。

具体表现在：

- Claude Code 具有强交互感和深度推理，主动提问并解释思路
- Codex 以首次尝试准确率（first-attempt accuracy）著称，但速度稍慢
- Claude Code 会主动测试代码、修复环境问题
- Codex 倾向于完成实现后让用户自行安装依赖和运行

体验差异会被 AGENTS.md 削弱

如果你在 AGENTS.md 中明确指定了工作流程（如“实现前先与我确认方案”），两款代理的行为差异会显著缩小。不要过度依赖社交媒体上对差异的夸大描述。

### 5.2 市场数据

表 4: 市场采用数据（截至 2026 年 3 月）

| 指标           | Claude Code | Codex |
|--------------|-------------|-------|
| VS Code 安装量  | 6.1M        | 5.4M  |
| VS Code 评分   | 4/5         | 3.5/5 |
| GitHub Stars | 65–72K      | ~64K  |

### 5.3 本章小结

Claude Code 的交互感更强，Codex 在直接任务上的首次准确率更高。但通过 AGENTS.md 等配置文件可以大幅拉近两者的行为差异。市场数据显示 Claude Code 在安装量和评分上略占优势。

## 6 RAG Pipeline 实验对比

### 6.1 实验设计

作者设计了一个定量可评估的任务——构建一个针对学术论文的 RAG（Retrieval-Augmented Generation）问答 Pipeline：

1. 从 Hugging Face 每日论文中取 5 篇论文

- 2. 构建包含 100 个问题和标准答案的测试集
- 3. 给两个代理相同的 prompt，要求构建完整 RAG Pipeline

给两个代理的统一要求：

- Python 实现，使用 PyMuPDF 处理 PDF
- 自选分块策略（chunking strategy）
- 自选向量索引方案
- 使用 llama-3.1-8b-instant 生成答案
- 证据不足时返回 fallback 响应，不得幻觉

两个代理均使用最高配模型 (Opus 4.6 / GPT-5.3-Codex), High effort 推理强度, 无 AGENTS.md。

6.2 实现方案对比

表 5: RAG Pipeline 实现方案对比

| 维度           | Claude Code                                                                                                 | Codex                                         |
|--------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Embedding 模型 | all-MiniLM-L6-v2                                                                                            | all-MiniLM-L6-v2                              |
| Top-K        | $k = 5$                                                                                                     | $k = 5$                                       |
| 向量存储         | ChromaDB                                                                                                    | FAISS (更底层、内存效率更高)                            |
| 分块策略         | 递归字符分割: $\backslash n \backslash n \rightarrow \backslash n \rightarrow . \rightarrow$ 空格; 1000 字符/200 字符重叠 | 句子级词分割, 每块 220 词/40 词重叠                       |
| 检索度量         | 原始 L2 距离                                                                                                    | 内积 (cosine) 分数                                |
| 置信度机制        | 单阈值 (L2 > 1.2 视为不相关) + 平均距离检查                                                                               | 多标准三级机制: strong / moderate / insufficient     |
| 代码架构         | 扁平函数式, 模块级常量                                                                                                | OOP Pipeline 类, 集中配置, dataclass, argparse CLI |

工程质量观察

Codex 的实现在代码架构上明显更优：面向对象设计、集中配置管理、类型标注 (dataclass)、CLI 参数解析。在大型或正式项目中，这种工程化差异非常关键。

6.3 行为差异

- **Claude Code**: 自动端到端测试脚本，确保 Pipeline 开箱即用。首次运行无错误。
- **Codex**: 完成实现后要求用户手动 `pip install` 并运行脚本。首次运行报错，经修复后正常。
- Codex 解释计划时更加详尽 (verbose)，Claude Code 更倾向于直接写代码执行命令。
- Codex 首个 token 的响应延迟可达 1 分钟，Claude Code 明显更快。

6.4 评测结果

使用 GPT-5.4 作为 LLM-as-a-Judge，在 Correctness、Completeness、Relevance、Conciseness 四个维度评估：

表 6: RAG Pipeline 评测结果（100 个问题）

| 结果 | Claude Code 胜出 | Codex 胜出 | 平局 |
|----|----------------|----------|----|
| 题数 | 42             | 33       | 25 |

Claude Code 胜出的关键因素

Claude Code 胜出主要归因于：（1）更宽松的置信度阈值，避免了过多 fallback 响应；（2）稍高的生成温度（0.2 vs. Codex 的 0.1），增加了回答的丰富度。

实验局限性

这是一个简单的单次实验。在专业场景中，分块策略、向量数据库、检索方案等架构决策应由开发者主导，而非完全交给 AI。此实验更适合作为观察两款代理”默认行为”的参考，而非最终结论。

6.5 本章小结

在 RAG Pipeline 任务中，两款代理选择了相似的基础方案但在关键细节上各有侧重。Claude Code 在端到端交付和最终评测上略优，Codex 在代码工程质量上更胜一筹。

7 生态系统与定价

7.1 Anthropic 生态的吸引力

作者认为选择编程代理不仅是选工具，更是选生态：

- **Anthropic 生态**：Claude Chat + Claude Code + Claude Cowork，形成类似 Apple 的闭环体验。Anthropic 还在逐步构建 OpenClaw（主动式个人代理）的安全版本。
- **OpenAI 生态**：除 Codex 外，其他产品对作者吸引力不大。ChatGPT 在 UI、对话风格和模型选择上已落后于 Claude Chat。

7.2 定价方案对比

表 7: 定价方案对比

| 层级          | Claude Code | Codex   |
|-------------|-------------|---------|
| 入门版         | \$20/月      | \$20/月  |
| 中间版（Max 5x） | \$100/月     | —       |
| 重度使用版       | \$200/月     | \$200/月 |



### Claude Code 的定价优势

Claude Code 提供了 \$100/月的中间层级 (Max 5x), 对大多数开发者来说已经足够。这避免了从 \$20 直接跳到 \$200 的尴尬, 使其**实际使用成本更低**。

## 7.3 Skills 与 Plugins 生态

- Skills 在两个平台间兼容, 体验一致
- Codex 的 plugin 支持推出较晚, 可用插件较少
- 社区内容 (Reddit、X、博客) 以 Claude Code 为主, 反映其更大的社区规模
- 作者认为大多数开发者并不使用 plugins, 这不应成为决策的关键因素

## 7.4 本章小结

Anthropic 正在构建一个紧密整合的产品生态, 对于已经使用 Claude Chat 的用户吸引力尤其大。Claude Code 的三级定价比 Codex 的两级定价更灵活。Skills 生态基本通用, plugins 差异不大。

# 8 总结与延伸

## 8.1 作者的最终选择

作者选择回归 Claude Code 的两个核心原因:

1. **生态系统**: Anthropic 的 Claude Chat + Code + Cowork 整合体验优于 OpenAI
2. **定价灵活性**: \$100/月的中间层级更经济实用

## 8.2 关键结论

### 如何选择编程代理

1. 两者都不是“错误的选择”——模型能力接近, 都能完成日常编程任务
2. **任务类型**比 benchmark 更能决定哪个更适合你
3. 最可靠的方法: 用 \$20/月版本在自己的实际编程领域上测试可量化的任务
4. AI 工具格局每隔几个月剧变一次, 不要把当前选择视为永久决定

### 8.3 全文核心对比总览

表 8: Claude Code vs. Codex 核心维度总览

| 维度           | Claude Code          | Codex         |
|--------------|----------------------|---------------|
| 底层模型         | Opus 4.6             | GPT-5.3-Codex |
| 长任务可靠性 (50%) | 12 小时                | 5 小时 50 分     |
| Token 效率     | 较低 (3-4×)            | 较高 (基准)       |
| 执行速度         | 更快                   | 较慢            |
| 交互风格         | 主动交互、解释推理            | 任务导向、详述计划     |
| 端到端交付        | 自动测试、开箱即用            | 需用户手动验证       |
| 代码架构质量       | 扁平函数式                | OOP、工程化更强     |
| 生态整合         | Chat + Code + Cowork | 相对分散          |
| 中间定价         | \$100/月              | 无             |
| VS Code 评分   | 4/5                  | 3.5/5         |

### 8.4 拓展阅读

- 原文链接: Hesamation – Claude Code vs. Codex: The Definitive Guide
- @GergelyOrosz 对 Claude Code 和 Codex 开发者的访谈 (How Codex is Built)
- Morph 关于 Opus vs. Codex 的 token 经济性研究
- Task-Completion Time Horizon 基准测试